



Tecnológico nacional de México

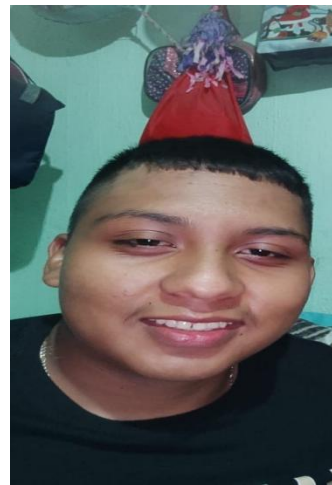
Instituto Tecnológico de Ciudad Guzmán

Carrera: Ingeniería en Sistemas Computacionales

Asignatura: LENGUAJES Y AUTOMATAS I

Nombre de la actividad: Práctica 1: Análisis léxico

Nombre de los alumnos:



Rafael Cervantes de la Cruz Alexis Alejandro Solis Santos

Docente: Marco Antonio Guzmán Solano

Grupo / Semestre: 6to A

Lugar y fecha de entrega: Ciudad Guzman, Jalisco. 27/04/2026

Índice

1.	Introducción.....	3
2.	Marco teórico.....	3
2.1	Lenguajes Formales y Compiladores.....	3
2.2	Análisis Léxico.....	3
2.3	Expresiones Regulares (E.R.).....	4
2.4	Autómata Finito Determinista (AFD).....	4
2.5	Gramáticas formales.....	6
3.	Desarrollo.....	6
1.	Descripción del lenguaje RAFI.....	6
1.1	Estructura general del programa.....	6
1.2	Tipos de datos y variables.....	6
1.3	Instrucciones de control.....	6
1.4	Instrucciones de Entrada / Salida.....	6
2.	Expresiones regulares.....	7
3.	Autómatas finitos deterministas (AFD).....	7
4.	Gramática del lenguaje RAFI.....	8
5.	Autómatas Finitos Deterministas (AFD).....	9
6.	Implementación	12
4.	Conclusiones.....	14
	Conclusión 1.....	14
	Conclusión 2.....	14
5.	Referencias.....	15

Práctica 1: Análisis léxico

1. Introducción

El presente documento constituye el reporte técnico de la Práctica 1: Analizador Léxico para el lenguaje de programación RAFI. El analizador léxico constituye la primera fase de un compilador: su función es recorrer el código fuente carácter a carácter y agrupar las secuencias en unidades denominadas tokens, que representan los elementos básicos del lenguaje (palabras reservadas, identificadores, operadores, literales, etc.).

El sistema fue implementado en Java y genera tres archivos de salida: un archivo .dep con el código fuente depurado (sin comentarios), un archivo .tab con la tabla de símbolos de las variables declaradas, y un archivo .tok con la clasificación completa de todos los tokens encontrados. El objetivo principal es comprender los fundamentos teóricos del análisis léxico, expresiones regulares, autómatas finitos y gramáticas formales y aplicarlos en un sistema funcional.

Objetivo

Diseñar e implementar un analizador léxico funcional para el lenguaje RAFI.

2. Marco Teórico

2.1 Lenguajes Formales y Compiladores

Un lenguaje formal es un conjunto de cadenas de símbolos definido sobre un alfabeto finito mediante reglas precisas. Los compiladores transforman programas escritos en un lenguaje de alto nivel a código ejecutable a través de varias fases secuenciales: análisis léxico, análisis sintáctico, análisis semántico, generación de código intermedio, optimización y generación de código objeto.

2.2 Análisis Léxico

El análisis léxico (o scanning) es la primera fase del compilador. Lee la secuencia de caracteres del programa fuente y la divide en tokens. Cada token tiene un tipo (categoría), un lexema (cadena real en la fuente) y, en algunos casos, un valor asociado. El analizador léxico también elimina comentarios y espacios en blanco, simplificando el trabajo del analizador sintáctico.

Los componentes fundamentales de un token son:

- **Nombre (nombre de categoría):** Identifica la clase del token, p.ej. PALABRA_RESERVADA, IDENTIFICADOR, ARITMETICO.
- **Tipo (subtipo):** Especifica el token dentro de su categoría, p.ej. PROG, ID, SUMA.
- **Lexema:** La cadena exacta del código fuente, p.ej. 'rafi', 'MiPrograma', '+'.
- **Línea:** Número de línea donde aparece el token, para reporte de errores.

2.3 Expresiones Regulares (E.R.)

Las expresiones regulares son patrones que describen conjuntos de cadenas sobre un alfabeto. Se construyen a partir de símbolos básicos mediante operadores: concatenación (yuxtaposición), alternancia (|) y cerradura de Kleene (*). Son la herramienta formal para especificar los patrones de los tokens en el análisis léxico.

Token	Expresión regular	Descripción
Identificador	letra (letra dígito _)*	Comienza con letra; continúa con letras, dígitos o _
Constante entera	dígito dígito* (= dígito+)	Secuencia de uno o más dígitos decimales
Cadena literal	" cualquier_caracter* "	Cualquier texto entre comillas dobles
Asignación	:=	Dos caracteres ':' seguido de '='
Comentario	/* cualquier_caracter* */	Todo entre /* y */ (incluyendo saltos de línea)
Operadores arit.	+ - * /	Cualquiera de los cuatro operadores aritméticos
Agrupación	({ [] { } [])	Paréntesis, llaves o corchetes
Punto y coma	;	Delimitador de sentencias
Coma	,	Separador de variables en declaraciones

2.4 Autómata Finito Determinista (AFD)

Un Autómata Finito Determinista (AFD) es una quintupla $M = (Q, \Sigma, \delta, q_0, F)$ donde Q es el conjunto finito de estados, Σ es el alfabeto de entrada, $\delta: Q \times \Sigma \rightarrow Q$ es la función de transición, $q_0 \in Q$ es el estado inicial y $F \subseteq Q$ es el conjunto de estados de aceptación. La propiedad determinista garantiza que para cada estado y símbolo existe exactamente una transición definida.

En el análisis léxico, cada patrón de token se implementa como un AFD. El analizador léxico actúa como una máquina de estados que, al leer un carácter, cambia de estado según δ . Cuando alcanza un estado de aceptación (F), ha reconocido un token.

Definición formal de AFD

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q : Conjunto finito de estados

Σ : Alfabeto de entrada (conjunto finito de símbolos)

δ : Función de transición $\delta: Q \times \Sigma \rightarrow Q$

q_0 : Estado inicial ($q_0 \in Q$)

F : Conjunto de estados de aceptación ($F \subseteq Q$)

La propiedad fundamental del AFD es que para cada par (estado, símbolo) existe a lo sumo una transición definida, lo que garantiza el determinismo en el procesamiento. El autómata acepta una cadena si, partiendo del estado inicial q_0 y procesando todos los símbolos, termina en un estado de

aceptación (estado final). A continuación, se presentan los AFD principales del lenguaje RAFI:

AFD-1: Identificadores y Palabras Reservadas

El AFD para identificadores tiene 3 estados. El estado q0 es el estado inicial. Al leer una letra transita a q1 (estado de aceptación parcial). Desde q1, si se lee letra, dígito o guión bajo, permanece en q1. Cualquier otro carácter lleva al estado de rechazo q_err. Al finalizar en q1, el lexema acumulado se consulta en la tabla de palabras reservadas; si existe, el token es PALABRA_RESERVADA, de lo contrario es IDENTIFICADOR.

Estado	Letra	Dígito	_ (guión bajo)	Otro	¿Acepta?
q0 (inicio)	→ q1	q_err	q_err	q_err	No
q1	→ q1	→ q1	→ q1	retrocede	Sí (ID/PR)
q_err	---	---	---	---	Error

AFD-2: Constantes Enteras

El AFD de constantes enteras tiene 2 estados operativos. Desde q0, al leer un dígito pasa a q1 (aceptación). Desde q1, nuevos dígitos mantienen el estado en q1. Cualquier carácter no dígito termina el reconocimiento del token CENT.

Estado	Dígito (0-9)	Otro	¿Acepta?
q0 (inicio)	→ q1	q_err	No
q1	→ q1	retrocede	Sí (CENT)

AFD-3: Cadenas Literales

El AFD de cadenas literales inicia en q0. Al leer "" pasa a q1 donde acumula cualquier carácter excepto "". Al encontrar la segunda "" pasa al estado q2 (aceptación), generando el token CLIT.

Estado	"	Cualquier char (no ")	\n	¿Acepta?
q0 (inicio)	→ q1	q_err	q_err	No
q1	→ q2	→ q1	→ q1	No
q2	---	---	---	Sí (CLIT)

AFD-4: Operador de Asignación :=

El AFD de asignación reconoce la secuencia de dos caracteres ':='. Desde q0, ':' lleva a q1; si el siguiente carácter es '=' alcanza q2 (aceptación del token ASIG). Cualquier otro carácter en q1 genera un error léxico.

Estado	':'	'='	Otro	¿Acepta?
q0 (inicio)	→ q1	q_err	q_err	No
q1	q_err	→ q2	error	No
q2	---	---	---	Sí (ASIG)

2.5 Gramáticas formales

Una gramática formal $G = (VN, VT, P, S)$ consta de: VN (variables o no terminales), VT (terminales), P (producciones) y S (símbolo inicial). La jerarquía de Chomsky clasifica las gramáticas en cuatro tipos; el análisis léxico opera con gramáticas de Tipo 3 (regulares), que producen lenguajes regulares reconocibles por AFDs.

Para RAFI, la gramática que define la estructura léxica (tokens individuales) es de tipo 3, mientras que la gramática que define la estructura completa de un programa válido es de tipo 2 (libre de contexto), procesada por el análisis sintáctico.

3. Desarrollo

1. Descripción del lenguaje RAFI

1.1 Estructura general del programa

Todo programa en RAFI sigue la siguiente estructura obligatoria:

```
rafi <NombrePrograma>    /* Encabezado: palabra clave + nombre */

decl                      /* Sección de declaraciones */
    <tipo> <var1>, <var2>; /* Una o más declaraciones de variables */

inicio                    /* Inicio del cuerpo del programa */
    <sentencias>          /* Instrucciones ejecutables */
fin                        /* Fin del programa */
```

1.2 Tipos de datos y variables

Tipo	Keyword	Valor por defecto	Descripción
Entero	entero	0	Números enteros con signo
Cadena	cadena	" "	Texto entre comillas dobles
Booleano	booleano	false	Valor lógico verdadero/falso

1.3 Instrucciones de control

- **Condicional:** si (<condición>) entonces ... fin
- **Condicional con alternativa:** si (<cond>) entonces ... sino ... fin
- **Ciclo mientras:** mientras (<condición>) hacer ... fin

1.4 Instrucciones de Entrada / Salida

- **impdig(<var>):** imprime el valor numérico de una variable entero.
- **impcad("texto"):** imprime una cadena literal en pantalla.
- **leerdig(<var>):** lee un dígito/entero del teclado y lo almacena en la variable.
- **leercad(<var>):** lee una cadena del teclado y la almacena en la variable.

2. Expresiones regulares del lenguaje

Las siguientes expresiones regulares formales definen todos los tokens reconocibles por el analizador léxico de RAFI. Se usa la notación estándar: + = una o más repeticiones, * = cero o más, | = alternancia, [] = clase de caracteres, () = agrupación.

#	Token	Identificador	Expresión Regular Formal
1	Identificador	ID	[a-z A-Z] ([a-z A-Z 0-9 _])*
2	Constante entera	CENT	[0-9]+
3	Cadena literal	CLIT	" ([^" \n]) * "
4	Asignación	ASIG	: = (dos caracteres consecutivos)
5	Suma	SUMA	\+
6	Resta	RESTA	\-
7	Multiplicación	MULTIPLICACION	*
8	División	DIVISION	\/
9	Par. abierto	PARENTESIS_ABIERTO	\(
10	Par. cerrado	PARENTESIS_CERRADO	\)
11	Llave abierta	LLAVE_ABIERTA	\{
12	Llave cerrada	LLAVE_CERRADA	\}
13	Corch. abierto	CORCHETE_ABIERTO	\[
14	Corch. cerrado	CORCHETE_CERRADO	\]
15	Punto y coma	PC	;
16	Coma	COMA	,
17	Comentario	(elim.)	/ * ([^*] * [^/]) * * /
18	Palabra reservada	PR	rafi decl entero cadena booleano inicio fin ...

3. Autómatas Finitos Deterministas (AFD)

Se presentan las tablas de transición completas de los AFDS que reconocen los tokens más representativos del lenguaje RAFI. Los estados marcados con * son estados de aceptación (estados finales).

AFD General del Analizador Léxico

El analizador implementa implícitamente un AFD global que, partiendo del estado inicial q0, determina qué sub-AFD activar según el primer carácter leído:

Primer carácter	AFD activado	Token generado
Letra [a-zA-Z]	AFD-1 (Identificador/PR)	ID / PALABRA_RESERVADA
Dígito [0-9]	AFD-2 (Constante entera)	CENT
"	AFD-3 (Cadena literal)	CLIT
':'	AFD-4 (Asignación :=)	ASIG
/ seguido de *	AFD-5 (Comentario)	(descartado)
+ - * /	Reconocimiento directo	SUMA, RESTA, etc.

() { } []	Reconocimiento directo	PARENTESIS, etc.
;,	Reconocimiento directo	PC, COMA
Otro	Sin AFD válido	ERROR LÉXICO

4. Gramática del lenguaje RAFI

La gramática libre de contexto (Tipo 2 de Chomsky) que define la estructura sintáctica completa de un programa RAFI se presenta en notación BNF (Backus-Naur Form). Los símbolos en mayúsculas son no terminales; los símbolos en minúsculas y entre comillas son terminales.

/* Gramática BNF del lenguaje RAFI */

<programa> ::= 'rafi' ID <declaraciones> <cuerpo>

<declaraciones> ::= 'decl' <lista_decl>
| épsilon

<lista_decl> ::= <decl_var> <lista_decl>
| épsilon

<decl_var> ::= <tipo> <lista_ids> ';'

<tipo> ::= 'entero' | 'cadena' | 'booleano'

<lista_ids> ::= ID
| ID ',' <lista_ids>

<cuerpo> ::= 'inicio' <sentencias> 'fin'

<sentencias> ::= <sentencia> <sentencias>
| épsilon

<sentencia> ::= <asignacion>
| <condicional>
| <ciclo>
| <entrada_salida>

<asignacion> ::= ID ':=' <expresion> ';'

<condicional> ::= 'si' '(' <expresion> ')' 'entonces' <sentencias> 'fin'
| 'si' '(' <expresion> ')' 'entonces' <sentencias>
'sino' <sentencias> 'fin'

<ciclo> ::= 'mientras' '(' <expresion> ')' 'hacer' <sentencias> 'fin'

```

<entrada_salida> ::= 'impdig' '(' <expresion> ')' ';'
                  | 'impcad' '(' CLIT ')' ';'
                  | 'leerdig' '(' ID ')' ';'
                  | 'leercad' '(' ID ')' ';'

```

```

<expresion> ::= <termino> <expresion_prima>

```

```

<expresion_prima> ::= '+' <termino> <expresion_prima>
                  | '-' <termino> <expresion_prima>
                  | epsilon

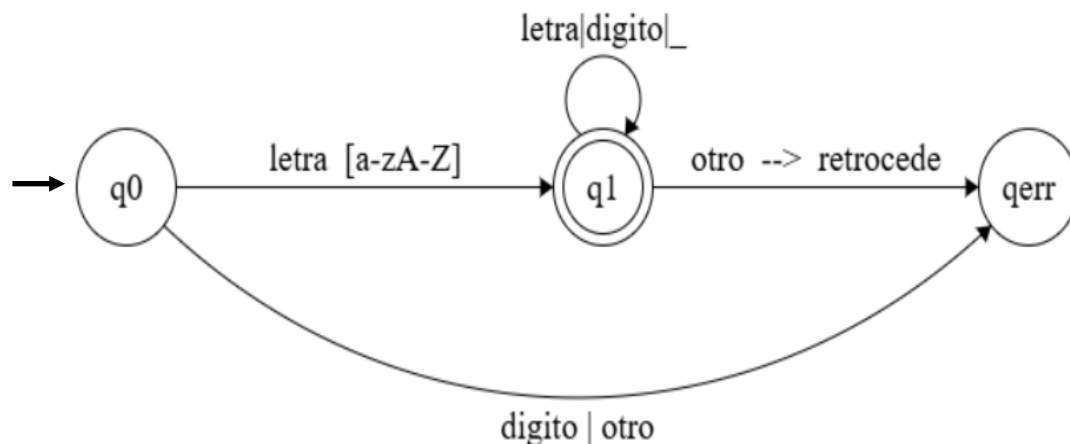
```

5. Autómatas Finitos Deterministas (AFD)

Los siguientes diagramas representan los Autómatas Finitos Deterministas (AFDs) del analizador léxico de RAFI. Cada estado se representa con un círculo; los estados de aceptación se muestran con doble círculo y borde verde. La flecha de entrada indica el estado inicial q0. Las etiquetas sobre las flechas indican el carácter o clase de caracteres que produce cada transición.

AFD-1: Identificadores y Palabras Reservadas

Reconoce identificadores (ID) y palabras reservadas (PR). Comienza en **q0**. Al leer una letra transita a **q1** (estado de aceptación). Desde q1 se permanece con letra, dígito o guión bajo (_). Al terminar en q1 el lexema se consulta en la tabla de reservadas: si existe es PALABRA_RESERVADA; si no, es IDENTIFICADOR. El estado **qerr** captura cualquier entrada inválida.

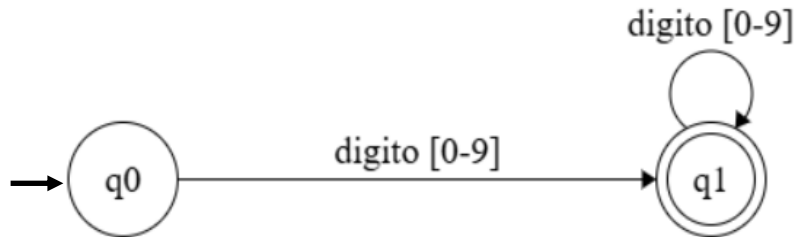


→ q0	Estado inicial — espera primera letra
q1*	Estado de aceptación — lexema válido (ID o PR)
Qerr	Estado de error — carácter no permitido en este contexto

AFD-2: Constantes Enteras (CENT)

Reconoce secuencias de uno o más dígitos decimales (0-9). Desde el estado inicial **q0**, al leer un dígito transita al estado de aceptación **q1**. Desde q1,

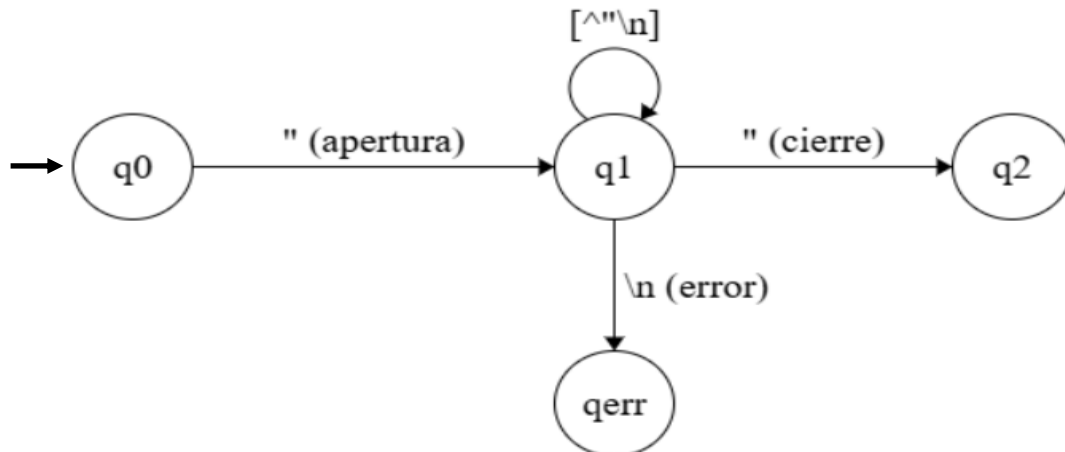
nuevos dígitos mantienen el estado. Cualquier carácter no-dígito termina el token CENT mediante retroceso (el carácter no se consume).



→ q0	Estado inicial
q1*	Acepta: token CENT generado. Retrocede ante no-dígito

AFD-3: Cadenas Literales (CLIT)

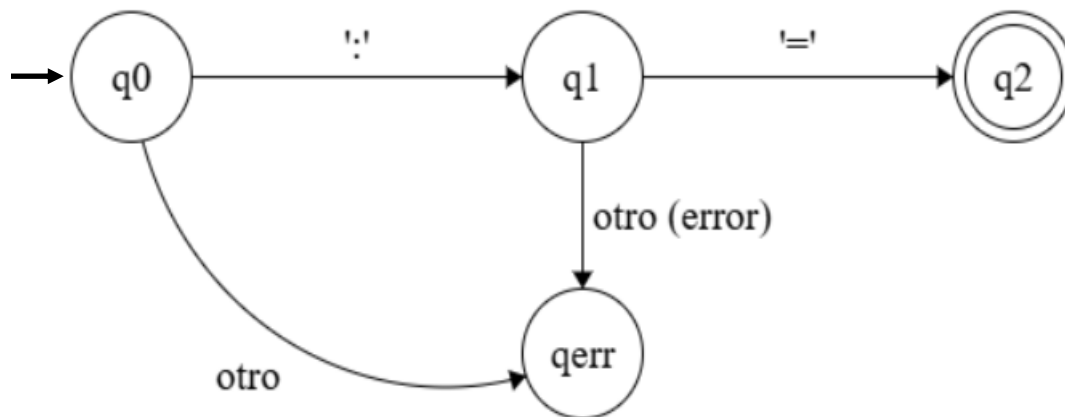
Reconoce cadenas de texto delimitadas por comillas dobles. Desde **q0**, la comilla de apertura " lleva a **q1** donde se acumula cualquier carácter excepto comilla y salto de línea. La comilla de cierre " lleva a **q2** (aceptación). Un salto de línea dentro de la cadena lleva a **qerr** (cadena no terminada).



→ q0	Estado inicial — espera comilla de apertura
q1	Dentro de la cadena — acumula caracteres
q2*	Acepta: token CLIT generado
qerr	Error: cadena no terminada (salto de línea)

AFD-4: Operador de Asignación (:=)

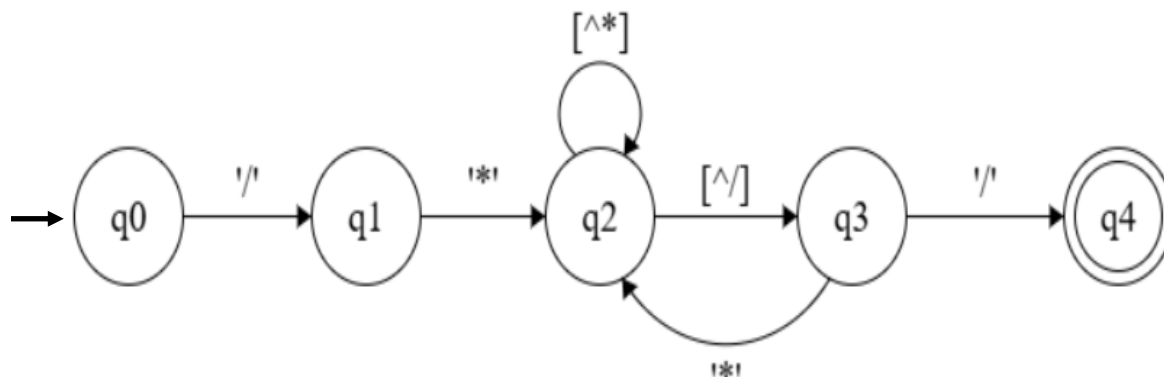
Reconoce la secuencia de dos caracteres :=. Desde **q0**, el carácter ':' lleva a **q1**. Si el siguiente carácter es '=' se alcanza q2 (aceptación del token ASIG). Cualquier otro carácter en q1 que no sea '=' conduce a **qerr** (error léxico: ':' aislado).



→ q0	Estado inicial
q1	Leyó ':' — espera '='
q2*	Acepta: token ASIG (:=) generado
qerr	Error: carácter inesperado después de ':'

AFD-5: Comentarios (/ * ... */)

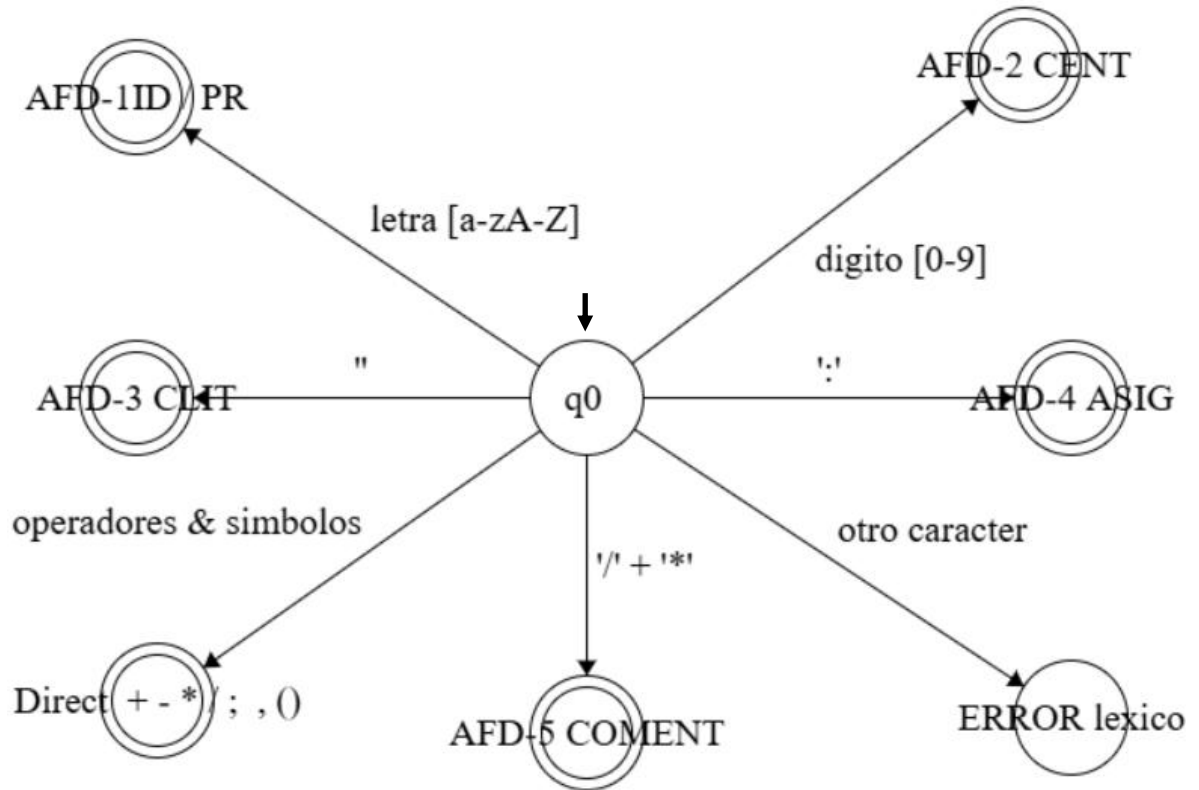
Reconoce y descarta bloques de comentario. Desde **q0**, '/' lleva a **q1**. Si el siguiente es '*' entra en **q2** (cuerpo del comentario). En q2 cualquier carácter se mantiene en q2, excepto '*' que lleva a **q3** (posible cierre). En q3, '/' cierra el comentario llegando a **q4** (aceptación/descarte). Si en q3 el carácter no es '/' se regresa a q2.



→ q0	Estado inicial — espera '/'
q1	Leyó '/' — espera '*' para comenzar comentario
q2	Dentro del comentario — acumula cualquier carácter
q3	Posible fin: leyó '*' — espera '/'
q4*	Acepta: comentario completo (se descarta, no genera token)

AFD General del Analizador Léxico

El analizador léxico implementa implícitamente un **AFD** general que actúa como despachador: desde el estado inicial **q0**, el primer carácter leído determina cuál sub-AFD se activa. El diagrama siguiente muestra las transiciones del estado inicial q0 hacia cada sub-autómata.



6. Implementación

6.1 Arquitectura del Sistema

El sistema está compuesto por tres clases Java que se comunican de la siguiente manera:

Clase	Responsabilidad	Entradas / Salidas
Token.java	Modelo de datos para un token (nombre, tipo, lexema, línea).	Salida: objeto Token con los cuatro atributos.
AnalizadorLexico.java	Implementa el AFD global; recorre la fuente y produce tokens.	Entrada: String (código fuente). Salida: List<Token> + lista de errores.
Main.java	Orquesta el flujo completo: lectura, depuración, análisis y escritura de archivos.	Entrada: progfte.txt. Salida: .dep, .tab, .tok.

6.2 Clase Token.java

La clase Token es el modelo de datos básico del sistema. Contiene cuatro atributos públicos y un constructor que los inicializa. El método toString() devuelve una representación legible para depuración.

```
public class Token {
    public String nombre;    // Categoría: PALABRA_RESERVADA,
                             // IDENTIFICADOR, etc.
    public String tipo;      // Subtipo: PROG, ID, SUMA, CENT, etc.
    public String lexema;    // Texto original del código fuente
    public int linea;        // Número de línea donde aparece

    public Token(String nombre, String tipo, String lexema, int linea)
    { ... }
}
```

6.3 Clase AnalizadorLexico.java

Es el núcleo del sistema. Implementa el método **analizar(String entrada)** que recorre el código fuente mediante un índice entero *i* y una variable *linea*. El reconocimiento se realiza con una cadena de condicionales if-else que corresponde directamente al AFD general:

- Paso 1: Si el carácter es '\n', incrementa el contador de línea.
- Paso 2: Espacios, tabuladores y '\r' se ignoran (tokens blancos).
- Paso 3: '/*' activa el sub-AFD de comentarios (consume hasta encontrar '*/').
- Paso 4: ':' seguido de '=' genera token ASIG.
- Paso 5: Letra → sub-AFD de identificador/palabra reservada. Consulta tabla RESERVADAS.
- Paso 6: Dígito → sub-AFD de constante entera; acumula mientras sean dígitos.
- Paso 7: '"' → sub-AFD de cadena literal; acumula hasta encontrar el cierre.
- Paso 8-11: Operadores, agrupación, ; y , → reconocimiento directo de un carácter.
- Paso 12: Cualquier otro carácter → error léxico registrado con número de línea.

6.4 Clase Main.java

La clase Main coordina el proceso completo. Sus responsabilidades se dividen en cinco métodos privados más el método main:

- **main()**: lee progfte.txt, llama a depurar(), instancia AnalizadorLexico, llama a construirTablaSimbolos(), y genera los tres archivos de salida.
- **depurar()**: elimina comentarios /* */, colapsa espacios múltiples, elimina líneas vacías. Produce el contenido de progfte.dep.

- **construirTablaSimbolos():** recorre la lista de tokens buscando DECL e INICIO; entre ambos capturan variables con su tipo y línea. Produce el Map que alimenta progfte.tab.
- **generarContenidoTabla():** formatea la tabla de símbolos como texto alineado.
- **generarArchivoTok():** genera el archivo .tok con cuatro secciones: tabla de símbolos, lista de tokens, resumen por categoría y errores léxicos.

6.5 Archivos de Salida

Archivo	Contenido	Propósito
progfte.dep	Código fuente sin comentarios, normalizado.	Verificar que la depuración funciona correctamente.
progfte.tab	Tabla de variables con tipo, valor por defecto y línea.	Base para la tabla de símbolos del compilador.
progfte.tok	Tabla de símbolos + lista de tokens + resumen + errores.	Resultado principal del análisis léxico.

4. Conclusiones

Conclusión 1

Realizar esta práctica me permitió entender de forma más clara cómo funciona la primera fase de un compilador. Antes solo conocía las expresiones regulares y los autómatas finitos de manera teórica, pero al programar el analizador léxico en Java pude ver cómo esos conceptos se convierten en código real que reconoce los tokens del lenguaje RAFI.

Me pareció interesante cómo el analizador no solo identifica palabras reservadas, identificadores y operadores, sino que también detecta y reporta los errores léxicos indicando el renglón exacto donde ocurren. Con esto concluyo que el análisis léxico es una fase fundamental del compilador, ya que sin una correcta identificación de tokens no es posible avanzar a las siguientes etapas.

Conclusión 2

El desarrollo del analizador léxico para el lenguaje RAFI representó mi primer acercamiento práctico y concreto a una de las etapas más fundamentales dentro de la construcción de un compilador. Uno de los aprendizajes más importantes fue entender la diferencia entre un lexema y un token. El lexema es el texto tal como aparece en el código fuente, por ejemplo, la palabra inicio, mientras que el token es la representación formal que el compilador utiliza internamente, en este caso INICIO. Esta distinción, aunque parece sutil, es fundamental porque permite que el analizador sintáctico trabaje con representaciones abstractas y no con cadenas de texto directas. Con esta fase de desarrollo he concluido que detrás de algo tan normal como escribir código hay una maquinaria pulida y bien definida que permite al computador entender lo que el programador escribe.

5. Referencias bibliográficas

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compiladores: Principios, técnicas y herramientas* (2.^a ed.). Pearson Educación.
2. Mogensen, T. AE. (2010). *Introducción a la compilación de compiladores*. Universidad de Copenhague. (Traducción al español disponible en repositorios académicos)
3. Carrillo Mendoza, J. (2012). *Teoría de la computación: Lenguajes, autómatas, gramáticas*. Alfaomega Grupo Editor.
4. Brookshear, J. G. (2012). *Ciencias de la computación: Una visión general* (11.^a ed.). Pearson Educación.